

Compte rendu

Atelier Professionnel 2

MediatekDocuments — Application C# / API REST PHP

Formation	BTS SIO — Option SLAM
Langages	C# / PHP / MySQL

Sommaire

1. Contexte	3
2. Outils et technologies utilisés	3
3. Réalisation des missions	4
3.1. Mission 0 — Installation et configuration	4
3.2. Mission 2 — Gestion des commandes	4
3.2.1. Tâche 1 — Commandes de livres et DVD	5
3.2.2. Tâche 2 — Commandes de revues	6
3.3. Mission 4 — Authentification et droits d'accès	7
3.4. Mission 5 — Sécurité, qualité et logs	8
3.5. Mission 6 — Tests et documentation	9
3.6. Mission 7 — Déploiement et sauvegardes	10
4. Bilan final	11

1. Contexte

La société InfoTech Services 86 (ITS 86) assure la gestion du parc informatique du réseau des médiathèques MediaTek86. Plusieurs types d'employés coexistent dans chaque médiathèque : le service Administratif (gestion des achats), le service Prêts (gestion des emprunts) et le service Culture (organisation d'événements).

Un premier développeur a mis en place la base de données et les fonctionnalités de consultation des documents. Ce deuxième atelier consiste à faire évoluer l'application de bureau C# et l'API REST PHP associée, en ajoutant la gestion des commandes, l'authentification, la sécurité, les tests et le déploiement.

Le travail a été réalisé en solo. Les missions 1 et 3 étant optionnelles pour un développeur seul, elles n'ont pas été réalisées.

Dépôt C#	https://github.com/flut0/mediatekdocuments
Dépôt API REST	https://github.com/flut0/rest_mediatekdocuments
API déployée	Alwaysdata (voir README pour le mode opératoire)

2. Outils et technologies utilisés

Technologie / Outil	Rôle
C# / Visual Studio 2022	Application de bureau Windows
PHP 8 / API REST	Couche d'accès aux données via HTTP
MySQL / phpMyAdmin	Base de données relationnelle
Postman	Tests fonctionnels de l'API
SonarQube for IDE	Analyse qualité du code C#
GitHub	Gestion de versions et pull requests
Alwaysdata	Hébergement de l'API et de la BDD
WampServer	Environnement local PHP/MySQL
NLog	Gestion des logs applicatifs
Newtonsoft.Json	Sérialisation/désérialisation JSON en C#

3. Réalisation des missions

Suivi Kanban

Un projet Kanban a été créé dans le dépôt GitHub mediatekdocuments. Les issues ont été créées pour chaque tâche et déplacées de 'To do' vers 'In Progress' au démarrage puis vers 'Done' à la fin. Toutes les missions ont été complétées.

To Do (départ)	In Progress	Done
M6 T1 : Tests (5h) M6 T2 : Docs (1h) M7 T2 : Sauvegardes (1h) M Bilan (6h)	M4 : Auth + droits (4h) M5 T1 : Sécurité (2h) M5 T2 : SonarQube (1h) M5 T3 : Logs (1h)	M2 T1 : Commandes livres/DVD (8h) M2 T2 : Commandes revues (4h) M6 T3 : Vidéo (2h) M7 T1 : Déploiement (3h)

3.1 Mission 0 — Installation et configuration

Avant de démarrer le développement, les outils suivants ont été installés et configurés :

- Visual Studio 2022 avec les extensions SonarQube for IDE et Newtonsoft.Json
- WampServer pour l'environnement PHP/MySQL local
- NetBeans pour l'analyse et le développement de l'API REST
- Postman pour les tests fonctionnels de l'API
- Récupération de l'API REST et de l'application C# depuis les dépôts d'origine
- Création de la BDD mediatek86 via phpMyAdmin (import du script SQL fourni)
- Création de 2 dépôts GitHub (mediatekdocuments et rest_mEDIATEKDOCUMENTS)
- Création d'un projet Kanban GitHub Projects avec toutes les issues

3.2 Mission 2 — Gestion des commandes

La mission 2 est la plus volumineuse de l'atelier. Elle ajoute la gestion complète des commandes de documents (livres, DVD et revues) dans l'application C#.

3.2.1 Tâche 1 — Commandes de livres et DVD

La gestion des stades d'une commande n'étant pas encore modélisée, une table Suivi a été créée pour stocker les différentes étapes : en cours, relancée, livrée, réglée. La table CommandeDocument a été modifiée pour inclure une clé étrangère vers Suivi.

```
CREATE TABLE suivi (id INT PRIMARY KEY AUTO_INCREMENT, libelle VARCHAR(50) NOT NULL);
```

Deux onglets ont été créés (livres et DVD) avec une logique identique. Chaque onglet permet :

- Sélectionner un document et afficher sa liste de commandes
- Saisir et enregistrer une nouvelle commande (étape initiale : 'en cours')
- Modifier l'étape de suivi selon les règles métier
- Supprimer une commande si elle n'est pas encore livrée

Règles métier : une commande livrée ou réglée ne peut pas revenir à une étape précédente. Une commande ne peut être réglée que si elle est déjà livrée. Ces règles sont gérées dans la méthode

GetSuivisAffiches.

Deux triggers ont été créés : un pour la génération automatique des exemplaires lors du passage à l'étape 'livrée' (numéro séquentiel, état 'neuf'), et un pour la suppression en cascade dans la table fille.

3.2.2 Tâche 2 — Commandes de revues

Un onglet dédié aux commandes de revues (abonnements) a été créé :

- Sélection d'une revue et affichage de ses abonnements (triés par date décroissante)
- Saisie d'un nouvel abonnement (date commande, montant, date de fin)
- Suppression uniquement si aucune parution n'est rattachée à l'abonnement
- Alerte au démarrage : liste des revues expirant dans moins de 30 jours

La méthode ParutionDansAbonnement vérifie si une date de parution est comprise entre la date de début et la date de fin d'un abonnement. Un test unitaire a été créé sur cette méthode.

3.3 Mission 4 — Authentification et droits d'accès

Une fenêtre d'authentification a été ajoutée comme première fenêtre au démarrage. Deux tables ont été créées dans la BDD :

```
CREATE TABLE service (id INT PRIMARY KEY AUTO_INCREMENT, libelle VARCHAR(50));  
  
CREATE TABLE utilisateur (login VARCHAR(50) PRIMARY KEY, pwd VARCHAR(100), idService INT,  
FOREIGN KEY (idService) REFERENCES service(id));
```

Service	Droits d'accès
Administratif	Toutes les fonctionnalités
Administrateur	Toutes les fonctionnalités
Prêts	Consultation uniquement (lecture seule)
Culture	Accès refusé — message d'information puis fermeture

L'alerte d'abonnement expirant n'est affichée qu'aux utilisateurs du service administratif et à l'administrateur.

3.4 Mission 5 — Sécurité, qualité et logs

Tâche 1 — Problèmes de sécurité

Deux problèmes de sécurité ont été identifiés et corrigés via des issues GitHub avec pull request :

Problème 1 — Le couple login:pwd d'accès à l'API était en dur dans le code de la classe Access. La correction consiste à déplacer ces informations dans App.config et à les récupérer via ConfigurationManager.AppSettings.

Problème 2 — L'accès à l'URL racine de l'API sans paramètres affichait la liste des fichiers. La correction consiste à modifier le fichier .htaccess pour retourner une erreur 400.

Tâche 2 — Contrôle qualité SonarQube

SonarQube for IDE a été configuré dans Visual Studio. Le code ajouté a été nettoyé : suppression des variables inutilisées, respect des conventions de nommage (PascalCase pour les méthodes, sauf les méthodes événementielles), gestion des exceptions.

Tâche 3 — Intégration des logs

NLog a été intégré pour enregistrer les logs dans un fichier. Dans la classe Access, chaque affichage console a été doublé d'un appel au logger :

```
private static readonly Logger logger = LogManager.GetCurrentClassLogger();  
  
logger.Info("Récupération des livres réussie");  
  
logger.Error("Erreur lors de la récupération : " + e.Message);
```

3.5 Mission 6 — Tests et documentation

Tâche 1 — Tests unitaires et tests Postman

Des tests unitaires ont été écrits sur les classes du package Model, notamment sur la méthode ParutionDansAbonnement. Une collection de tests Postman a été construite pour contrôler toutes les fonctionnalités de l'API REST (GET, POST, PUT, DELETE sur chaque ressource).

Tâche 2 — Documentation technique

Les commentaires XML ont été vérifiés et complétés. La documentation technique de l'application C# a été générée via Visual Studio et celle de l'API PHP via phpDocumentor sous NetBeans. Les deux documentations ont été transférées dans les dépôts GitHub.

Tâche 3 — Vidéo de démonstration

Une vidéo de démonstration d'environ 10 minutes a été créée, présentant l'ensemble des fonctionnalités de l'application C# : authentification, gestion des documents, commandes de livres/DVD, commandes de revues et alertes d'abonnement.

3.6 Mission 7 — Déploiement et sauvegardes

Tâche 1 — Déploiement

L'API REST PHP et la base de données ont été déployées sur Alwaysdata :

- Modification du fichier .env pour les informations de connexion à la BDD de production
- Upload des fichiers via SFTP (FileZilla)
- Import du script SQL via phpMyAdmin sur Alwaysdata
- Test de l'API déployée avec Postman
- Modification du code C# pour pointer vers l'URL de l'API distante
- Création d'un installateur Windows (Setup Project Visual Studio) et transfert sur GitHub

Tâche 2 — Sauvegardes automatiques

Une tâche planifiée a été configurée sur Alwaysdata pour une sauvegarde quotidienne automatique de la base de données à 02h00 via mysqldump.

4. Bilan final

Objectifs atteints

Mission	Objectif	Statut
Mission 0	Installation et configuration	Atteint
Mission 2 T1	Commandes livres/DVD + triggers	Atteint
Mission 2 T2	Commandes revues + alertes	Atteint
Mission 4	Authentification + droits	Atteint
Mission 5 T1	2 problèmes de sécurité corrigés	Atteint
Mission 5 T2	Nettoyage code SonarQube	Atteint
Mission 5 T3	Logs NLog intégrés	Atteint
Mission 6 T1	Tests unitaires + Postman	Atteint
Mission 6 T2	Documentation technique	Atteint
Mission 6 T3	Vidéo de démonstration	Atteint
Mission 7 T1	Déploiement API + installeur	Atteint
Mission 7 T2	Sauvegardes automatiques	Atteint

Compétences couvertes

Compétence	Missions
Exploiter les ressources du cadre applicatif (C#, PHP)	M2, M4, M5
Identifier, développer et adapter des composants logiciels	M2, M4
Utiliser des composants d'accès aux données (API REST, MySQL)	M2, M4, M5
Intégrer en continu (GitHub, pull requests, branches)	M5
Réaliser des tests (unitaires C#, fonctionnels Postman)	M6
Rédiger des documentations technique et d'utilisation	M6, Bilan
Déployer un service (API + installeur)	M7
Gérer des sauvegardes automatiques	M7
Mettre en place des authentifications et habilitations	M4
Évaluer la qualité d'une solution applicative (SonarQube)	M5